Functional & Non-Functional Requirements

Functional Requirements

FR1 Role-Based Access with Admin-Provisioned Accounts

The system shall support four roles (Student, Staff, Facility Manager, Maintenance Personnel) enforced via ASP.NET Core Identity. Facility Managers shall create all accounts through the desktop application; no public self-registration is permitted. Every Web API endpoint shall enforce role authorization before executing any action.

FR2 Facility and Facility Type Management

Facility Managers shall be able to create, edit, and deactivate facility types (e.g., Lab, Classroom, Meeting Room, or any custom type) and individual facilities from the desktop application. Each facility shall record a name, type, capacity, and location, persisted through the Web API to the shared SQL Server database.

FR3 LINQ-Based Facility Availability Search

Students and Staff shall search for available facilities from the web application by date, hourly time slot (08:00–20:00), facility type, and minimum capacity. The API shall execute these filters as a single LINQ query over the DbContext, returning results sorted by facility name and availability.

FR4 Facility Reservation with Conditional Approval Flow

Students and Staff shall reserve facilities by selecting a facility and one or more hourly slots on a chosen date. Reservations for Classrooms and Meeting Rooms shall be auto-confirmed if a LINQ-based conflict check finds no overlap; reservations for Labs shall be created in Pending status until a Facility Manager explicitly approves or rejects them on the desktop.

FR5 Booking Cancellation and Modification Window

Users shall be able to cancel or modify their own reservations up to 2 hours before the slot start time. Cancellations shall immediately free the slot in the database, and a delegate-based notification shall inform the Facility Manager of the change.

FR6 Delegate-Based In-App Notification System

The solution shall define a shared NotificationHandler delegate type, raised on five events: booking confirmed, booking approved/rejected, booking cancelled, maintenance assigned, maintenance status changed. The same delegate mechanism shall drive toast popups + an inbox list in the WinForms desktop and equivalent banners + an inbox page in the MVC web application, with no duplicated event logic.

FR7 Maintenance Issue Reporting with Photo Attachment

Students and Staff shall submit maintenance issue reports from the web application, providing a target facility, description, severity (Low/Medium/High/Critical), and an optional photo upload. The photo shall be transmitted as a multipart form and stored on the API server, with the file path recorded in the database.

FR8 Manual Maintenance Task Assignment

Facility Managers shall view all open issue reports on the desktop application and manually assign each one to a specific Maintenance Personnel user. Upon assignment, the notification delegate shall immediately fire an alert to the assignee and record an audit entry with the timestamp and assigning manager.

FR9 Maintenance Status Tracking and CSV Log Generation

Maintenance Personnel shall update each assigned task through the states Pending → In Progress → Resolved from the desktop application, with every transition timestamped. Facility Managers shall generate maintenance logs filtered by facility, date range, status, and assignee using LINQ aggregation; the resulting log shall be displayed in a grid and exportable to CSV through a FileStream-based writer.

FR10 Desktop Resource Scheduler Dashboard

The desktop application shall provide a day-and-week calendar grid (facilities on one axis, hourly time slots on the other) rendering all confirmed bookings and scheduled maintenance windows, color-coded by status. The grid shall support LINQ-backed filters for facility type, status, and date range, and shall refresh automatically every 30 seconds plus on demand via a Refresh button.

Non-Functional Requirements

NFR1 Performance

Under typical academic load (≤100 facilities, ≤1,000 active bookings, ≤20 concurrent users), every LINQ-backed search, availability check, and log aggregation shall return within 2 seconds end-to-end, and each Web API call shall complete within 1 second of server processing time.

NFR2 Security

All passwords shall be stored as salted hashes via ASP.NET Core Identity (never plaintext). The Web API shall require a valid bearer token on every non-anonymous endpoint, reject requests failing role authorization with HTTP 403, and serve all production traffic over HTTPS.

NFR3 Data Integrity and Reliability

The database schema shall enforce a uniqueness constraint on the combination (FacilityId, Date, TimeSlot) so that concurrent reservation attempts cannot double-book the same slot; conflicting writes shall return HTTP 409 and surface a clear message to the user. Desktop and web views shall reflect any committed change within 30 seconds.

NFR4 Real-Time Data Consistency Across Platforms

The shared SQL Server database shall serve as the single source of truth for all bookings, maintenance records, facility data, and user accounts. Any change committed through one application (web or desktop) shall be retrievable through the Web API by the other application within 30 seconds, with no manual synchronisation step, fulfilling the “seamless data management and real-time updates” requirement of the project specification.

NFR5 Auditability of Bookings and Maintenance Actions

Every booking creation, approval or rejection, cancellation, maintenance issue report, maintenance assignment, and status transition shall be persisted with the acting user’s ID and a UTC timestamp, retained for the full academic term, and retrievable by a Facility Manager through the Web API for display and CSV export in the desktop log generator.